

stating boolean is correct or not. If it is false then...
 Boolean bool3 = new Boolean("Asha ka babar")

① कौन सा object को print करेगी तो to string() call होगा

Chapter 15: java.lang package

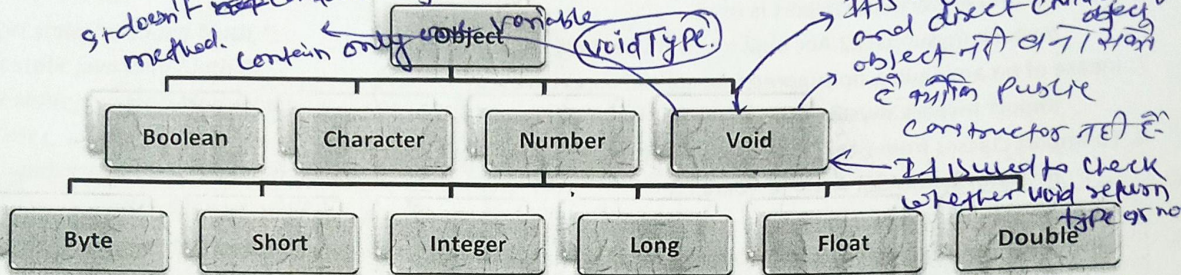
- Wrapper Classes
- String, StringBuilder and StringBuffer
- Object Class
- Runtime Class
- Thread Class

← wrap primitive into object form

Boolean b = new Boolean("True") ⇒ true
 "TRUE" ⇒ true

Wrapper Classes → Dynamic

to define several utility method which required for primitive



List of Primitive Data Types and its Wrapper Classes

Primitive Data Types	Wrapper Classes	Constructors
byte	Byte	byte or String
short	Short	short or String ('Byte')
int	Integer	int or String
long	Long	long or String
float	Float	float, double or String
double	Double	double or String
boolean	Boolean	boolean or String
char	Character	Character

NOTE: Void is also referred as wrapper class does not any public constructor.

* All wrapper classes are immutable.

Example: WrapperConstructors.java

Methods of Wrapper Classes

1. valueOf

- All wrapper classes except Character have this method.
- This method converts String to corresponding Wrapper objects
- It is a static method
- In case format of String is not correct it forwards NumberFormatException
- It can take, one(String) or two(String,int)* arguments

Float f = Float.valueOf("123.45");

Integer n1 = Integer.valueOf("123456");

Integer n2 = Integer.valueOf("1011010", 2); // Binary Format

Integer n3 = Integer.valueOf("12345677", 8); // Octal Format

उपर हम अपना define करेंगे तो बता सकेंगे

* Only with Integer and Long

इसका object के लिए आया - अर्थात् जो default use कर रहे हैं उसे अगर कि एक बता नहीं सकते है due to hashCode() ← call

Wrapper to Primitive "Auto unboxing"

Primitive to Wrapper "Auto boxing"

In all wrapper classes `toString()` is overridden to return `String` value.
- By: In all wrapper classes, `equals()` method is overridden.

Integer n4 = Integer.valueOf("4343543", 16); // Hexa Format

→ Every class except Character class. } for Content Comparison.

2. `parseXxx` where Xxx can be any numeric (primitive) data types

- Converts String to primitive data types
- It is a static method
- It is present in all numeric Wrapper Classes
- Generates `NumberFormatException` in case of invalid format

`parseXxx(String s, int radix)`

↳ Integer wrapper class

(Byte, Short, Integer, Long)

to convert char into string

`String.valueOf(words.charAt(i))`

`boolean b = Boolean.parseBoolean("true");`

`int num = Integer.parseInt("1234");`

`double dob = Double.parseDouble("1234");`

3. `toString`

Variant One

- It converts wrapper class objects to String
- It is a method available in all wrapper classes
- It does not take any argument

`Float f = new Float(10.23);`

`String str = f.toString();`

`Character c = new Character('a');`

`String str1 = c.toString();`

Variant Two

- All numeric wrapper classes provide this variant
- This method takes primitive data types and converts it to String
- It is a static method

`String str = Integer.toString(10);`

`String str1 = Double.toString(10.5);`

Variant Three

- This variant is only present in Integer and Long wrapper classes
- This is a static method that takes 2 arguments (primitive data type and radix)

`String s1 = Integer.toString(1478, 10);`

// Decimal

`String s2 = Integer.toString(1245, 16);`

// Hexadecimal

`String s3 = Integer.toString(1245, 2);`

// Binary

के लिए लिखेंगे।

4. `toXxxString`, where Xxx can be Binary, Octal, or Hex

- This method is present only in Integer and Long Wrapper Classes
- This is a static method
- This method takes int or long primitive data types and converts to Binary String, Octal String or Hex String

`String s1 = Integer.toHexString(65);`

`String s2 = Long.toBinaryString(65);`

`byteValue()`

`shortValue()`

`intValue()`

`longValue()`

`floatValue()`

`doubleValue()`

Every numerical wrapper class contains 6 methods

Wrapper के लिए नए लिखना बंद कर दिया।

Integer, Byte, Short, Long, Float, Double

for every object a unique number is generated by JVM which is nothing but hashcode. hashcode won't represent address of object. JVM will use hashcode while saving objects into hashing related data structure like HashTable, HashMap, HashSet etc.

Earlier versions of Java

```
Integer n = new Integer(10);
int n1 = n.intValue();
n1++;
n = new Integer(n1);
```

The main advantage of saving object based on hashcode is search operation will become easy. The most powerful search algorithm - till upto today is hashing.

Autoboxing and Auto unboxing (Java 5)

```
int x = 25;
Integer n = x;           // autoboxing (primitive to wrapper)
n++;                     // unwrap, increment, wrap
Integer x = 10;          // boxing
int y = x;                // unboxing (wrapper to primitive)
```

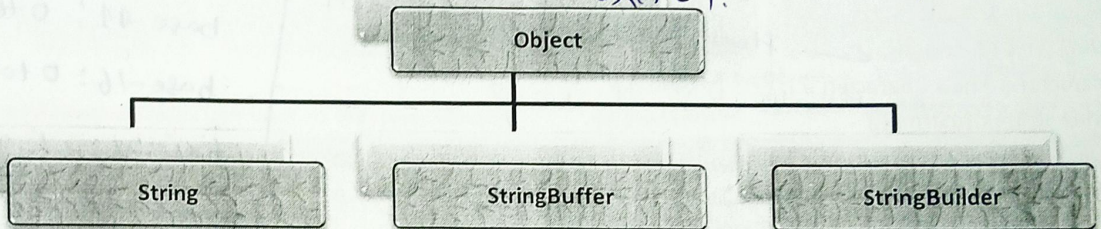
Integer I = 10;
↓ after compilation
Integer I = Integer.
valueOf(10);
↑
Compiler replaces it.

String, StringBuilder and StringBuffer

with the help of
xxx.valueOf()

↑ Compiler replace
array 1.

↑
Compiler replaces
it.



String

- It is immutable (Every operation you perform a new object will be created)

→ default capacity = 16 → formula → capacity = (current capacity + 1)

StringBuffer and StringBuilder

- It is mutable (Operation is performed on the same object)
- StringBuilder provides similar functionality like StringBuffer, but the methods are faster as these methods are non-synchronized.

Methods of StringBuffer/StringBuilder Class

append(String s)

reverse()

charAt(int index)

setCharAt(int index, char c)

insert(int index, String str)

deleteCharAt(int index)

delete(int index, int endIndex)

String substring(int startIndex)

String substring(int startIndex, int endIndex*)

int length()

int capacity()

* endIndex is exclusive

Examples: TestString.java and TestStringBuilder.java

indexOf(string str)
→ index of first occurrence of str in this string

If two object are true by == then equals also always returns true

* If $r_1.equals(r_2)$ is false then $r_1 == r_2$ is false

Object Class

- Object is the base class for all the classes
- Comes from java.lang package

Constructor

Object()

Operations (never change)

wait()

notify()

notifyAll()

Class getClass()

boolean equals(Object)

Object clone()

Overridables

String toString()

boolean equals(Object)

int hashCode()

finalize()

equals() only applicable for object type.

It is possible for both primitive type and object type.
== → we cannot override it because this is operator. bcoz this is operator. operators are always X

(operation as well as overridable)

(operation as well as overridable)

(point, employee)

java 3140 class hi equal ko override karne hai

	equals()	==	Example
Custom Class	If not overridden checks the memory, can be overridden to check data	Checks for memory rather than data	TestObject1.java TestObject2.java
String created using new Example: String str = new String("A");	Is overridden and checks data	Checks memory	TestObject3.java
String created using = Example: String str = "A";	Checks data	Checks data	TestObject3.java
Wrapper Object created using new Example: Integer i1 = new Integer(9);	Is overridden and checks data	Checks memory	TestObject4.java
Wrapper Object created using = Example: Integer i1 = 9;	Checks data	Checks data if < 128 else checks memory	TestObject5.java

equals contract

The equals method implements an equivalence relation on non-null object references:

- It is **reflexive**: for any non-null reference value x, x.equals(x) should return true.
- It is **symmetric**: for any non-null reference values x and y, x.equals(y) should return true if and only if y.equals(x) returns true.
- It is **transitive**: for any non-null reference values x, y, and z, if x.equals(y) returns true and y.equals(z) returns true, then x.equals(z) should return true.
- It is **consistent**: for any non-null reference values x and y, multiple invocations of x.equals(y) consistently return true or consistently return false, provided no information used in equals comparisons on the objects is modified.
- For any non-null reference value x, x.equals(null) should return false.

java 3140 class hi equal ko override karne hai

Primitive
Reference

==
equals

(==) → सामान्य जिन लक्षणों पर memory check करता है।

Note that it is generally necessary to override the hashCode method whenever this method is overridden, so as to maintain the general contract for the hashCode method

int hashCode()

The general contract of hashCode is:

- Whenever it is invoked on the same object more than once during an execution of a Java application, the hashCode method must consistently return the same integer, provided no information used in equals comparisons on the object is modified. This integer need not remain consistent from one execution of an application to another execution of the same application.
- If two objects are equal according to the equals(Object) method, then calling the hashCode method on each of the two objects must produce the same integer result.
- It is not required that if two objects are unequal according to the equals(java.lang.Object) method, then calling the hashCode method on each of the two objects must produce distinct integer results. However, the programmer should be aware that producing distinct integer results for unequal objects may improve the performance of hashtables.

Example: TestObject6.java

Runtime ① memory calculation ② execute exec

- The Runtime class encapsulates the run-time environment.
- We can only get a reference to the current Runtime object by calling the static method Runtime.getRuntime().

Runtime rt = Runtime.getRuntime(); → वही वस्तु है जिसका Runtime से create करने को होता है।

Methods

Process exec(String progName) throws IOException

Process exec(String comLineArray[]) throws IOException

long freeMemory()

void gc()

static Runtime.getRuntime()

long totalMemory()

String str = "Hello"

// Primitive

equals = defa

== data

Example: MemoryDemo.java

Example: ExecDemo.java

Example: ExecDemoFinis.java

Runtime का एक ही object बनता है।

Multithreading

Thread

- It is a smallest unit of execution
- It helps to share time from processor
- By default every application has single Thread, the "main" thread

Benefits of Multi Threading

1. Optimum utilization of resources (processor)
2. Performing more than 1 task simultaneously

garbage collector उसे कि मिला चमता है जिसकी गवा कर फेंक दे

किसी नि object को call करने पर / फ़िरवने पर testing
उम्मीद है।

- Better Functionality
- Better Performance

Multithreading VS Multitasking

Multithreading = Multitasking + Sharing of Data + Inter Thread Communication

States of Thread

There are 5 states of Thread.

1. **New:** Thread has just been created, neither is it standing in queue nor it is sharing time from processor
2. **Started/Runnable:** Standing in pool, waiting for its turn
3. **Running:** Executing and sharing time from processor
4. **Suspended/Blocked:** Temporarily withheld → *suspended for some time in order to satisfy some condition.*
5. **Dead:** Work assigned to thread is over, now it is not going to share time from processor

- To implement "Threading" in Java, Java provides "Thread" class
- This class comes from "java.lang" package
- There are 2 ways to create Thread
- 1. Create an object/implements Runnable
- 2. Inherit It *(extending Thread class).*

Runnable interface has one abstract method called run() method

Constructors of Thread class

Thread()
Thread(String threadName) *by default main*
Thread(Runnable run) *run कहे पर है।*
Thread(Runnable run, String threadName)

Runnable Interface

public void run()

This method defines running state of a Thread

Create an object

Example 1:

```
class Test implements Runnable {  
:  
    Thread t = new Thread(this);  
    // run function is present in current class  
:  
    public void run() {  
        // Statements  
    }  
}
```

guaranty thread के लिए
run current class में है।
Access का object बना रहे है।

implement मिरवने पर definition मिरवना compulsory है।

Example 2:

```
Thread t = new Thread(new A(), "mythread");
```

```
class A implements Runnable {  
    public void run() {
```

(Run in another class)

Thread को start किया जाता है उसको run में लाया जाता है

// Statements

}

}

Example 3:

```
Thread t = new Thread(new Runnable() {  
    public void run() {  
        // Statements  
    }  
}, "mythread");
```

Anonymous class

ये निम्नोक्त यहाँ पर override कर रहे हैं

Inherit It

```
class MyThread extends Thread {  
    public void run() {  
        // Thread  
    }  
}
```

recommended way

```
public void run() {  
    myThread t = new MyThread();  
    t.start();  
}
```

अपने आप Run call हो जायेगा

Data in Thread Class

1. Name

2. Priority (Order of execution): Range is from 1 to 10, default priority is Thread.NORM_PRIORITY i.e. 5

Thread.MAX_PRIORITY 10

Thread.MIN_PRIORITY 1

Thread.NORM_PRIORITY 5

3. Group/parent

4. Daemon

parent को स्वतंत्र होने पर child खुद खत्म हो जाये

Methods of Thread Class

```
String getName() / setName(String name)  
int getPriority() / setPriority(int priority)  
setDaemon(boolean) / boolean isDaemon()  
boolean isAlive()  
static Thread currentThread()  
int activeCount()
```

// false for dead state, true for other states

// returns the currently executing Thread

// returns the active number of Threads, apart from
Threads in New and Dead state

ThreadGroup

ThreadGroup creates a group of threads.

Constructors

ThreadGroup(String groupName)

ThreadGroup(ThreadGroup parentOb, String groupName)

Methods

int activeCount()

Static class & static data को access कर सकते हैं वही तो error देगा.

int activeGroupCount()

final void destroy()

int enumerate(Thread group[]) ← सारे के reference आ जाते हैं।

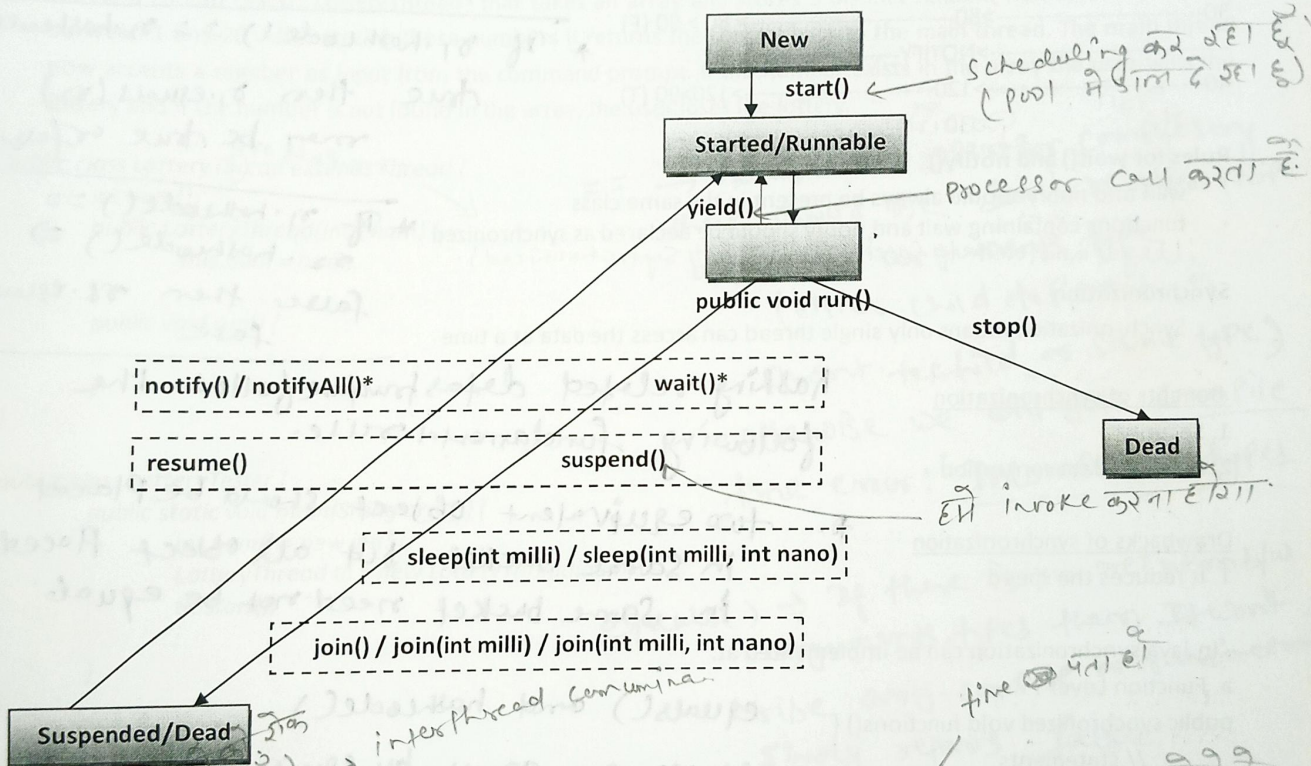
final int getMaxPriority()

final String getName()

final ThreadGroup getParent()

void list() ← Command Prompt पर दिखाना है।

State Transition Diagram



- Variants of sleep(), join() and wait() throws "InterruptedException"
- wait(), notify() and notifyAll() are the methods from Object class
- join makes the current thread wait, until the thread joined completes its task

Examples:

1. IntThread.java
2. TestMyThread.java
3. ThreadMethod1.java
4. ThreadMethod2.java
5. TestThread.java ←
6. TestThread1.java
7. Producer.java, Consumer.java, and Stock.java
8. DeadLockDemo.java ← दो को synchronized करे
9. ThreadGroupDemo.java

IllegalThreadStateException (Starting a single thread twice)

इसलिए जब कि equals() method को override करेंगे तो
hashCode को भी override करेंगे नहीं तो कोई error नहीं होगा पर यह **not a good programming practice** है।
vedisoft®
Software & Education Services Pvt. Ltd.

Producer	Stock	Consumer
0	5	
10	10	10 > 5 (T)
5	15	
20	25	25 > 15 (T)
10	90	
40	50	50 > 90 (F)
	NOTIFY	WAIT
30	80	80 > 90 (F)
	NOTIFY	WAIT
40	120	120 > 90 (T)

Rules for wait() and notify()

- wait and notify should always be present in the same class
- functions containing wait and notify should be declared as synchronized

illegal monitor state exception (if not synchronized).

Synchronization

- Synchronization means only single thread can access the data at a time

Benefits of synchronization

1. Security
2. It avoids data corruption

Drawbacks of synchronization

1. It reduces the speed

- In Java synchronization can be implemented at:

a. Function Level

```
public synchronized void functions() {
    // statements
}
```

b. Block Level

```
synchronized (this) {
    // statements
}
```

hashing related data structure follow the following fundamental rule.

- * two equivalent object should be placed in same bucket. but all object placed in same bucket need not be equal.

equals() and hashCode().

If two objects are equal by equals() method then their hashCode() must be equal. i.e. two equivalent object should have same hashCode.

If $r1.equals(r2)$ is true then $r1.hashCode() == r2.hashCode()$ is always true.

Assignment

Theory Questions

1. What are wrapper classes? How do they differ from primitive data types?
2. What are the differences between String, StringBuffer and StringBuilder?
3. What is thread? Explain its states.
4. How does multithreading differ from multitasking?
5. What is synchronization? How can we implement it in Java?
6. What is the difference between equals and ==?

whenever we are overriding
equals() compulsory we should override
to satisfy above contract. i.e. two equivalent objects
have same hashCode.

vedisoft
Software & Education Services Pvt. Ltd.

Module 1 - Desktop Technologies

String class में अगर कोई को String hashcode() कि true होगा पर बात यह है कि hashcode() → सारा Computer में same होगा क्योंकि hashcode() को एन्ट्रीज store है तो 3 बार।

Page 150

String के

Chapter 16: java.io package

- IO Basics
- File Class
- Character Streams - Readers and Writers
- Byte Streams - Streams and Serialization
- Rules for Serialization
- Console Class

IO Basics

- Java programs perform I/O through streams.
- A stream is an abstraction that either produces or consumes information.
- A stream is linked to a physical device by the Java I/O system. All streams behave in the same manner, even if the actual physical devices to which they are linked differ.

Byte Streams and Character Streams

- Java defines two types of streams: byte and character.
- Byte streams provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.
- Character streams provide a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.

The Predefined Streams

- System class contains three predefined stream variables: in, out, and err. These fields are declared as public, static, and final within System.
- System.out refers to the standard output stream. By default, this is the console.
- System.in refers to standard input, which is the keyboard by default.
- System.err refers to the standard error stream, which also is the console by default.
- These streams may be redirected as per our requirements to any compatible I/O device.

File Class

- Although most of the classes defined by java.io operate on streams, the File class does not. It deals directly with files and the file system.
- The File class does not specify how information is retrieved from or stored in files; it describes the properties of a file itself.
- A File object is used to obtain or manipulate the information associated with a disk file, such as the permissions, time, date, and directory path, and to navigate subdirectory hierarchies.

Constructors

File(String pathwithname)
File(String parent, String name)
File(File parent, String name)

NOTE: Creating an object of File class is not going to create the file.

Example:

File f = new File("C:\\Data\\First.java");

File → In
Byte Stream → new File()

bw.write("100");
bw.write("100");
bw.write("105");
bw.write("true");

overall performance
It's down because it's convert to string. So to overcome use

PrintWriters.
pw.write('d'); pw.write(10.5);
pw.write(100); pw.write(true);

pw.println(100)
↳ automatic \n

04/01/2024

Relative Path

```
File f = new File("C:/Data/First.java");  
File f1 = new File("C:\\Data", "First.java");  
File f2 = new File("C:\\Data");  
File f3 = new File(f2, "First.java");
```

./ First.java // Current
../ First.java // Parent
/ First.java // Root

Single backslash is illegal escape character compile error

Methods

```
String getName()  
String getParent()  
String getPath() (Path = Parent + Name)  
long length() // length in bytes  
long lastModified()
```

// 0 1 Jan 1970 00:00:00:000

How many character in the file

Example:

```
File f = new File("C:\\Data\\First.java");  
long lm = f.lastModified();  
java.util.Date d = new java.util.Date(lm);
```

human readable format convert kr de

```
boolean createNewFile() throws IOException  
boolean exists() // file or folder hai ya nahi  
boolean isFile() // data file hai ya na (physicist)  
boolean isDirectory() // directory file hai ya na  
boolean canRead()  
boolean canWrite()  
boolean isHidden()  
boolean isAbsolute()
```

// Archive
// Read Only

project ke andar kaise hai

Example:

```
File f = new File("First.java");  
boolean b = f.isAbsolute();  
File f1 = new File("C:\\Data\\First.java");  
boolean b1 = f1.isAbsolute();
```

//relative path
//false
//absolute path
//true

drive ke andar kaise hai

```
boolean delete() // directory file can be deleted only if it is empty  
boolean renameTo(File)  
boolean mkdir() // directory banane hai  
String[] list() // this methods all the contents of directory file  
boolean setReadOnly()  
String getAbsolutePath()  
File getAbsoluteFile()
```

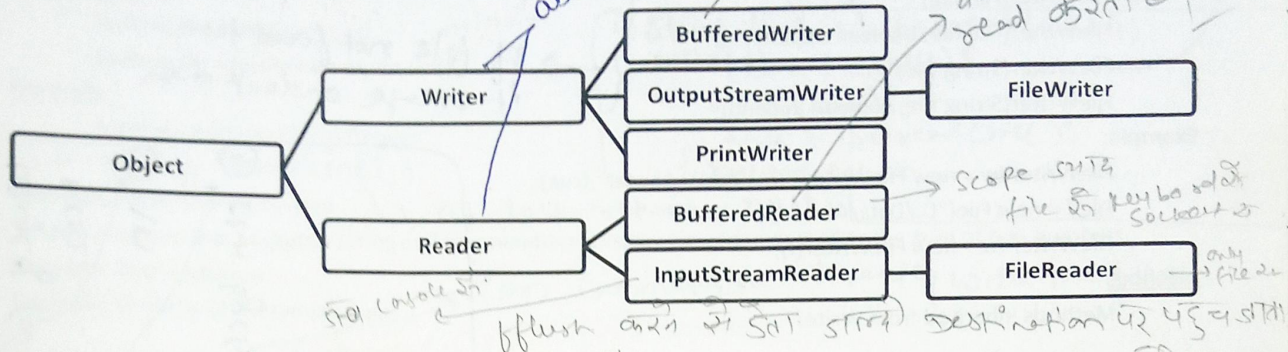
list out all file and directory in specified directory

directory se kisi file ka path kaise hai

It list out all file and directory in specified directory.

Example: WriteAndRead1.java
Example: WriteAndRead2.java
Example: WriteAndRead3.java

Character Streams - Writers and Readers



Writer → Notepad me likhne ke liye use kiya jata hai

- It is an abstract class for writing to character streams
- Most subclasses will override some of the methods defined in Writer to provide higher efficiency, functionality or both.

Methods

write(char[] cbuf) → String ko likhne ke liye
 write(char[] cbuf, int off, int len) → String ke kisi hisse ko likhne ke liye
 write(int c) → ASCII code likhne ke liye
 write(String str)
 write(String str, int off, int len)
 flush()
 close()

Reader

- Java Reader is an abstract class for reading character streams.
- Most subclasses will override some of the methods defined in Reader to provide higher efficiency, functionality or both.

If there is no next character then it returns -1

Methods

int read() → Abhi value kya hai
 int read(char[] cbuf) → Array ko likhne ke liye
 int read(char[] cbuf, int off, int len) → yehi se kaha tak puchhe 10 digits ya 3 puchha to 3 return krta hai
 boolean markSupported()
 mark(int readAheadLimit)
 boolean ready()
 void reset()
 long skip(long n)
 void close()

FileWriter

- This class is use to write sequence of characters to a File

File f = new File("abc.txt");

if we write 100 as it then use
 fw.write("100");
 bw.write("100");

Java Module 1 - Desktop Technology
 FileReader (F)
 char[] ch = new char[1024];
 fr.read(ch);
 for(char c: ch) { s.o.p(c); }

int i = fr.read();
 while(i != -1) {
 s.o.p((char)i);
 i = fr.read();
 }

Abstract class ko object nahi bana sakte

override kr dete hai.

1,0 me pass krte true/false

Constructors

- FileWriter(File file)
- FileWriter(File file, boolean append)
- FileWriter(String file)
- FileWriter(String file, boolean append)

→ If file not found then it create and write data

Example:

```
FileWriter fw = new FileWriter("D:\\Data\\Abc.txt", true);  
File f = new File("D:/Data/Abc.txt");  
FileWriter fw = new FileWriter(f);
```

Methods

Methods inherited from Writer

NOTE: All variants of write, flush and close method throws IOException

FileReader

- It is used to read set of characters from a File

Constructors

- FileReader(File file)
- FileReader(String file)

Methods

Methods inherited from Reader

NOTE: All variants of read method throws IOException, if we are reading a file and file does not exists it throws FileNotFoundException

Example: WriteAndRead4.java

Example: WriteAndRead5.java

Example: WriteAndRead6.java

Example: FileWriterDemo.java

BufferedWriter

← It is used to write data to the file.

- It is used to write a single character, arrays and strings of data
- It performs fast operation
- The size of buffer might or might not be specified

Constructors

- BufferedWriter(Writer out)
- BufferedWriter(Writer out, int size)

Methods

- newLine()
- Methods inherited from Writer

(to insert a line separator).

BufferedReader

- It is used to read a single character, arrays and strings of data
- It performs fast operation

BW - bw = new BW("abc.txt"); X

BW bw = new BW(new File("abc.txt")); X

BW bw = new BW(new fw("abc.txt")); ✓

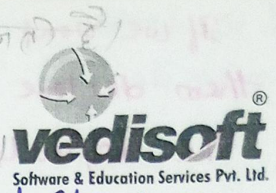
BW bw = new BW(new BW(new fw("abc.txt"))); ✓

Resource can be any class implement

file and object name is file not name

byte stream - secure is 311

space in name is 1



Do Buffered Readers both option are available read char by char or by line by line.

Constructors

BufferedReader(Reader in) *buffered*
BufferedReader(Reader in, int size)

Methods

Methods inherited from Reader

String readLine();

```
String line = br.readLine();  
while (line != null) {  
    s.o.p(line);  
    line = br.readLine();  
} br.close();
```

Example: FileReaderDemo.java *at last return null*

Write a program that accepts a string and two numbers. Display the string as well as the sum of two numbers.

Example: TestInput.java

Example: WriteTextFileExample.java *can communicate directly with the file*

PrintWriter *most advanced class* *most powerful - always use PrintWriters.*

- Java PrintWriter class is the implementation of Writer class.
- It is used to print the formatted representation of objects to the text-output stream.

File Readers for = new FileReader("abc.txt");
Buffered Readers br = new BufferedReader(fr);
br.close();

Constructor

PrintWriter(File file)
PrintWriter(File file, String charset)
PrintWriter(String fileName)
PrintWriter(String fileName, String charset)
PrintWriter(OutputStream out) *System.out*
PrintWriter(OutputStream out, boolean autoFlush)
PrintWriter(Writer out)
PrintWriter(Writer out)

internal file reader will be closed automatically always.

but for close() not close the br.

Methods

Methods inherited from Writer
print() *8 variant*
print(X)
println() *flush for endl*
println(X)
printf(String format, Object... args)
format(String format, Object... args)

Scanner for buffered reader
use and it

Example: PrintWriterExample.java

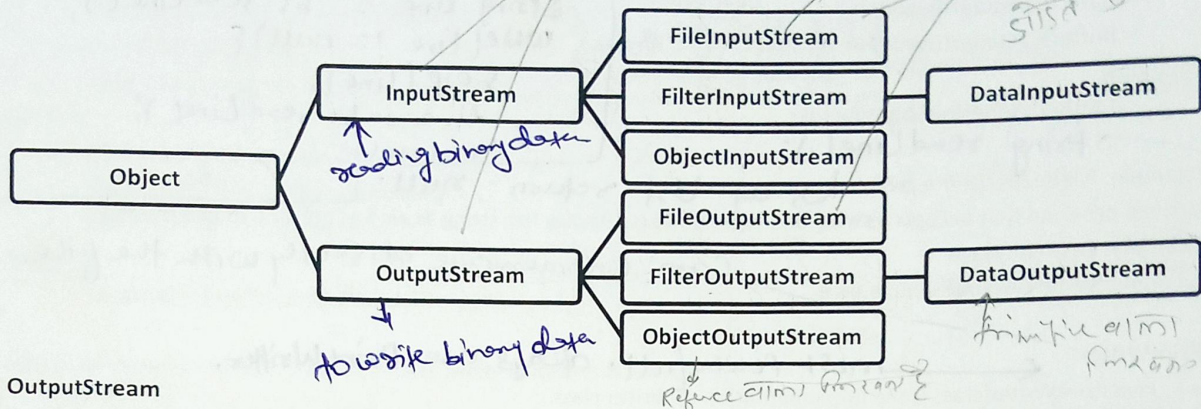
PrintWriter as Reference
variety of methods
variety of Data types
monitors as find try catch and monitor
file 43 Application monitor

File writer -> character by character format

buffer and char speed and balance and

If we want value of a variable not stored inside file then declare that variable with keyword **transient** so the default value is stored

Byte Streams - Streams and Serialization



OutputStream

- OutputStream class is an abstract class. It is the super class of all classes representing an output stream of bytes.
- Java application uses an output stream to write data to a destination, it may be a file, an array, peripheral device or socket.

Methods

write(int) throws IOException
write(byte[]) throws IOException
flush() throws IOException
void close() throws IOException

InputStream

- InputStream class is an abstract class. It is the super class of all classes representing an input stream of bytes.
- Java application uses an input stream to read data from a source, it may be a file, an array, peripheral device or socket.

Methods

int read() throws IOException
int available() throws IOException
close() throws IOException

FileOutputStream ← यदि file नहीं होगा तो बना देगा

- This class is used to perform simple file output operations.
- This class belongs to byte stream and stores data in the form of individual bytes

Constructors

FileOutputStream(String name)
FileOutputStream(String name, boolean append)
FileOutputStream(File file)
FileOutputStream(File file, boolean append)

Methods

Methods inherited from OutputStream

classCastException occur

Total data

Character data
(text data)

binary data
(video file, audio)

Readers / Writers -

file, images
Streams

transient is a modifier applicable only for variable.

Static int J = 20; } Both are not part of an object so it
 transient static int J = 20; } is not participate in serialization
 transient int i = 10; (it means J value not stored in file).
 In class if we print the value of J then it print "20" in each case from class level, not coming from file; print "0".

vedisoft
 Software & Education Services Pvt. Ltd.

FileInputStream

- This class is used to perform simple file input operations.
- This class belongs to byte stream and reads data in the form of individual bytes.

Constructors

FileInputStream(String name)
 FileInputStream(File file)

Methods

Methods inherited from InputStream

DataOutputStream

- This class provides function for writing binary I/O operations on primitive data types such as boolean, char, byte.

Constructors

DataOutputStream(OutputStream ins)

Methods

Methods inherited from OutputStream
 writeX(X) throws IOException

where X can be any primitive data types

Example

```
DataOutputStream dout = new DataOutputStream(new FileOutputStream("DataIO.txt"));
float price = 123.456f;
int code = 12;
dout.writeFloat(price);
dout.writeInt(code);
dout.flush();
dout.close();
```

↑
 ये file पर लिखेगा.

DataInputStream

- This class provides function for reading binary I/O operations on primitive data types such as boolean, char, byte.

Constructors

DataInputStream(InputStream ins)

Methods

Methods inherited from InputStream
 X readX(X) throws IOException

where X can be any primitive data types

Example

```
DataInputStream din = new DataInputStream(new FileInputStream("DataIO.txt"));
float price = din.readFloat();
int code = din.readInt();
din.close();
```

NOTE: The sequence and data types of writing and reading should be same otherwise Data corrupt (कॉरुप्ट)

Example: WriteAndReadData.java

Example: PrimitiveStreamExample.java

हमें 'a' value से Replace हो स्टार्टिंग
 जो JVM चोक करेगा, तो वह ये पढ़ेगा। Custom है
 at compile time
 Program int a = 10
 final int a = 10
 int b = 10;
 S.O.P(a) ⇒ S.O.P(10)
 S.O.P(b) ⇒ S.O.P(b);

Writing
user Define
Reading
Primitive

Serialize

Primitive

Deserialize

User Define

Serializable Interface

- This is an empty interface that does not define any method (marker interface)
- If a class implements Serializable Interface the object of this class can be written or read via a Stream
- If a class does not implement Serializable Interface and the object of this class is written or read via a Stream it generates NotSerializableException
- static and transient variables of class cannot be Serialized (they are used by programmers and not for storage)

ObjectOutputStream

- It is used to write objects(multiple data types) to streams

Constructor

ObjectOutputStream(InputStream)

Example

ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("DataIO.txt"));

Methods

Methods inherited from OutputStream

writeObject(<Object>) throws IOException

ObjectInputStream

- It is used to read objects(multiple data types) from streams

Constructor

ObjectInputStream(InputStream)

Example

ObjectInputStream in = new ObjectInputStream(new FileInputStream("DataIO.txt"));

Methods

Methods inherited from InputStream

Object readObject() throws IOException, ClassNotFoundException

Example: Book.java and ObjectStoreExample.java

Rules for Serialization

1. If a class does not implement Serializable Interface then the object of this class cannot be serialized or de-serialized. While serialization and de-serialization it will generate NotSerializableException.

Example: Book.java and Book1.java

1. transient cannot be serialized, incase of serialization and de-serialization it will hold default values numeric by 0, boolean by false, character by space and objects by null

Example: Book2.java

2. If base class implements Serializable then derive class automatically implements Serializable

Example: Employee.java

3. If derive class implements Serializable and base class does not implement Serializable. No Exception will be generated, but base class data will be as per the default constructor

Example: Employee1.java

4. Not only class, but all its instance variables needs to be Serialized

Example: Person.java and Person1.java

Singleton class

it is used to work on command prompt.
it is used to work on netbeans eclipse
does not work on netbeans eclipse
Null pointer exception

Console Class

- This is used to provide textual functionality to Console Application
- There are no Constructors

Console con = System.console();

Methods

```
String readLine()
String readLine(String fmt, Object... args)
char[] readPassword()
char[] readPassword(String fmt, Object... args)
Reader reader()
PrintWriter writer()
format(String fmt, Object... args)
```

Example: MyConsole.java

Example: MyConsole1.java

Assignment

Theory Questions

- What are the uses of File class. Explain its constructors and methods.
- Explain the Reader and Writer hierarchies.
- How are streams classified? Explain the various classes of streams hierarchy.
- What is Serialization and what are its benefits? How can we serialize a class.

Practical Questions

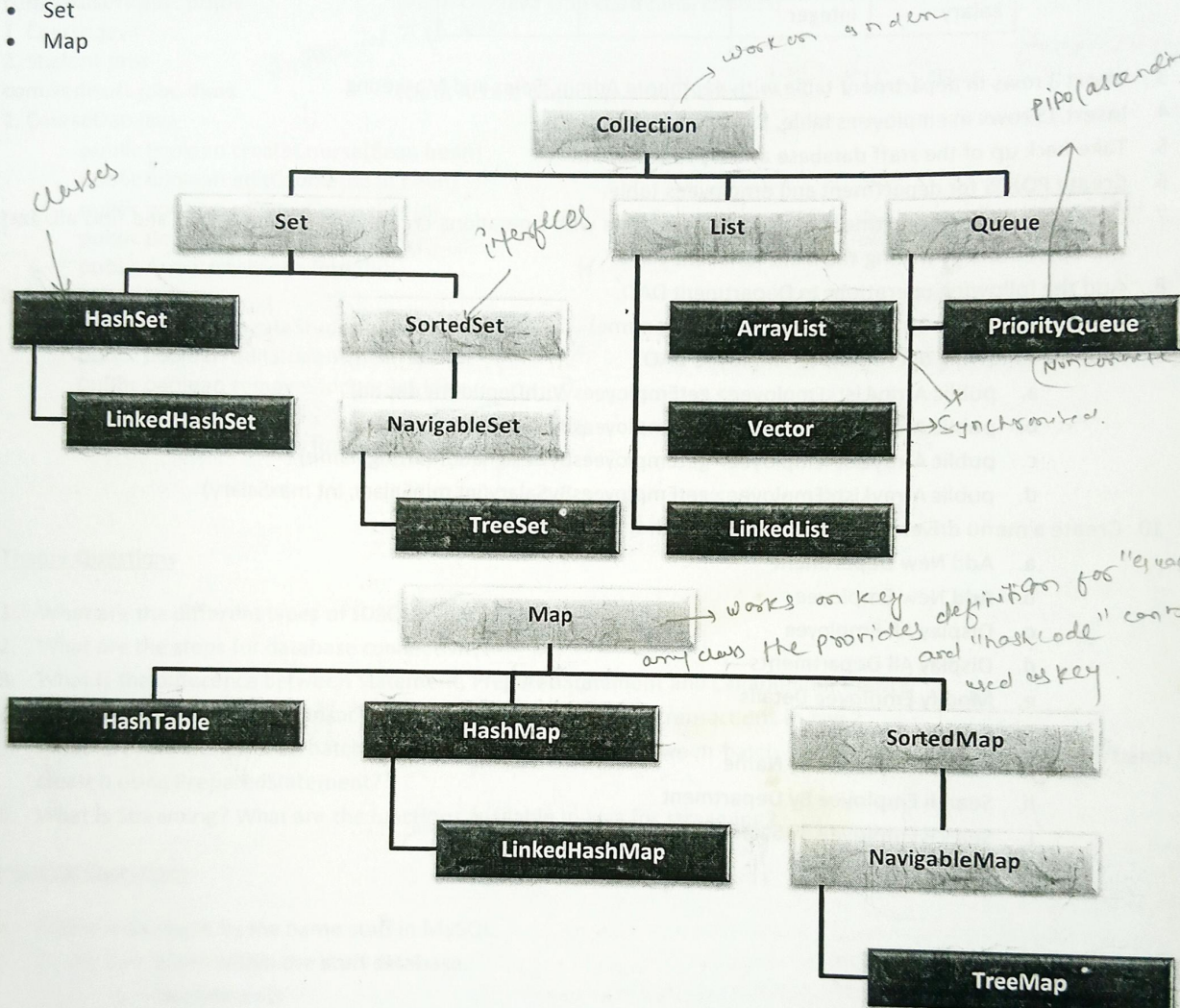
- Write a program that accepts a file name as a command line argument and check whether it exists or not, if it exists is an ordinary file display its name, parent, path, last modified date and time and length. If it is a directory file display its contents.
- Write a program that accepts a directory name as a command line argument and display its contents, also display the contents of its sub-directories.
- Write a program that accepts a directory name as command line argument and delete its contents also delete the contents of its sub directories.
- Write a program that accepts a directory name as command line argument and change all its contents to read only mode.
- Create your own implementation of "dir" command.
- Write a program that accepts 2 file names and copy the contents of one file to another. (using BufferedReader and BufferedWriter)
- Write a program that accepts a file name as a command line arguments and count the number of alphabets from a to z in that file.
- Write a program that accepts the name, phone number and address of 10 friends and store it in a File.
Hint: use ObjectOutputStream
- Write a program that displays the contents of File created in Question 8.
Hint: Use ObjectInputStream
- Write a program that accepts a friend's name, searches in the above created file and display its address and phone number. If the record is not found appropriate message should be displayed.

String name = in.readLine();

Set - No duplicate values (Non concrete)
 HashSet - unordered, unsorted.
 LinkedHashSet - insertion order.
 TreeSet - Ascending order

Chapter 18: java.util package

- Collection
- Map
- Sorting Collection and Arrays (Comparable and Comparator)
- Searching Collection and Arrays
- List
- Queue
- Set
- Map



Concrete Class/Map	Interface	Duplicate Elements	Ordered	Sorted	Methods invoked on Elements
ArrayList	List	Yes	By Index	No	equals()
Vector	List	Yes	By Index	No	equals()
LinkedList	List, Queue	Yes	By Index	No	equals()
HashSet	Set	No	No Order	No	equals() and hashCode()
LinkedHashSet	Set	No	By Insertion Order	No	equals() and hashCode()
TreeSet	Set	No	Sorted	Natural/Custom	equals() and compareTo()

Hash map - unordered, unsorted.
 Hash Table - unordered, unordered (synchronised).
 Linked hash map - insertion order
 Tree Map - Ascending order

HashMap	Map	Unique Keys	No Order	No	equals() and hashCode()
HashTable	Map	Unique Keys	No Order	No	equals() and hashCode()
LinkedHashMap	Map	Unique Keys	By Insertion Order or Last Accessed	No	equals() and hashCode()
TreeMap	SortedMap	Unique Keys	KeyOrder	Natural/Custom	equals() and compareTo()
PriorityQueue	Queue	Unique Keys	Sorted	To Do Order	equals() and hashCode()

List Interface

- It is used to store sequence of elements
- List Interface is implemented by ArrayList, Vector, LinkedList

ArrayList Class

- This class extends AbstractList class and implements List Interface
- Provides functionality for dynamic array (shrink and grow)
- Java 1.4 introduced the RandomAccess Interface, which the ArrayList implements

Vector Class

- Vector class is similar to ArrayList class, the vector methods are synchronized for thread - safety.
- Apart from ArrayList class, only the Vector class implements the RandomAccess Interface

LinkedList Class

- The LinkedList class extends the AbstractSequentialList class and implements the List Interface.
- Similar to an ArrayList, a LinkedList is an ordered list based on the index position of elements. Elements of a LinkedList are doubly linked to one another.
- Provides methods like addFirst(Object o), addLast(Object o)
- LinkedList iterate slowly as compared to ArrayList, however, it facilitates fast insertion and deletion of elements.
- With the release of Java 5, the LinkedList class has been enhanced to implement the Queue interface and supports methods like : peek(), poll() and offer()

The Set Interface

- Set Interface extends Collection
- It does not allow duplication of elements. If duplicate elements are added, add method returns false.
- equals method is used for comparison
- The classes that implements Set Interface HashSet, LinkedHashSet and TreeSet

The HashSet Class

- It extends AbstractSet class and implements Set Interface
- This class creates a collection using hashing mechanism. (hashCode is used as key to access data)
- It is unordered as well as unsorted with no duplicate values
- NOTE: if no order is required while iterating use HashSet

The LinkedHashSet Class

- It extends HashSet
- Maintains a linked list of elements in HashSet in the order they are inserted (doubly)
- NOTE: if order is required while iterating use LinkedHashSet

The TreeSet Class

- The TreeSet class implements the Set interface and stores elements in the form of a tree
- Arranges elements in Ascending order
- Used when fast retrieval is required and large size of data needs to be stored
- With Java 6, this class implements NavigableSet interface

The Queue Interface

- A Queue is a collection that holds elements before performing any operation
- Apart from collection, Queue provides operations, such as insertion, extraction, inspection.
- Principle used is FIFO
- Each queue shows the following nature: throws Exception when an operation fails

The PriorityQueue Class

- Introduced with Java 5, implements the Queue interface
- Arranged in natural order or Comparator interface(custom)
- The PriorityQueue class provides a Queue implementing the concept of priority-in and priority-out instead of FIFO

The Map Interface

- This interface maps a unique key to a specific value, and this key used to retrieve a value
- Chief Operations:
 - Locate a value based on a key
 - Retrieve all values of a Collection
 - Retrieve all keys of a Collection
- Classes that implements Map Interface are HashMap, Hashtable, LinkedHashMap and TreeMap

HashMap Class

- It uses Hash Table to implement the Map interface
- Implements Map and extends AbstractMap
- Provides unsorted and unordered Map
- It allows one null key, and numerous null values

LinkedHashMap Class

- Extends HashMap and implements Map interface.
- Maintains a linked list of elements within Map in the order in which they are inserted
- Elements are stored or removed slowly in comparison to HashMap
- Iteration is faster

HashTable Class

- Similar to Vector, HashTable also provides synchronized operations.
- Does not allow null key or a null value
- For storing and retrieving objects of a hashtable, the objects that are used as keys must implement the hashCode() and equals() methods

TreeMap Class

- Maintains data in sorted order by unique key and pair values
- Sorts the elements in ascending order

Example: TestCollection.java

Example: TestList.java

Example: TestSet.java

Example: TestQueue.java

Example: TestMap.java

Implementing Collections

1. Using the List Interface
2. Sorting Collections and Arrays
3. Searching Collections and Arrays
4. Converting Arrays into Lists and Vice Versa
5. Using the Map Interface
6. Navigating the TreeSet and TreeMap
7. Using the Set Interface
8. Using the PriorityQueue Class

Sorting Collections

- Method used is: Collections.sort(<Collection>)

Example: Sort1.java

natural order में कहा जाता है
Java अपन में परम से use करता है (यानि indendent करता है)

Using the Comparable Interface

- Looks for:
 public int compareTo(Object otherObject) → निम्नलिखित एन सुनिश्चित करता है Java.
 returns :
 -ve value if oneObject < otherObject
 +ve value if oneObject > otherObject
 0 value if oneObject = otherObject

Example: Sort2.java and Employee.java

Using the Comparator Interface

- Required when multiple sorting criteria's need to be mentioned
- Looks for :
 public int compare(Object oneObject, Object otherObject)
 returns :
 -ve value if oneObject < otherObject
 +ve value if oneObject > otherObject
 0 value if oneObject = otherObject
- Method used is: Collections.sort(<Collection>, <Comparator>)

return e1.age - e2.age

Example: Sort3.java

Sorting Arrays

- Method used is: Arrays.sort(<Array>)
- Method used is: Arrays.sort(<Array>, <Comparator>)

Example: Sort4.java

Example: TestSort.java

→ पहली sort करें फिर उसी collection में search करें

Searching Collections and Arrays

- `binarySearch()` method is used to perform search operations
- During successful searching, an index of the searched element, which can be positive or zero is returned.
- On unsuccessful searching, the int index of the insertion element is returned. (negative)
- Before searching an element, a collection or an array must be sorted
- If a collection or an array is sorted in a natural order, then it must also be searched in a natural order. Moreover, if a collection or an array is sorted using a `Comparator`, then it must also be searched using the same `Comparator`, which is passed as an argument within `binarySearch()` method.
- Methods used are:

`Collections.binarySearch(<Collection>, <Object element>)`

`Collections.binarySearch(<Collection>, <Object element>, <Comparator>)`

OR

`Arrays.binarySearch(<Array>, <Object element>)`

`Arrays.binarySearch(<Array>, <Object element>, <Comparator>)`

multiple values

Example: MySearchColl.java

Example: MySearchArr.java

head <, null
 tail >=, null
 sub >= <, null

Converting Lists to Arrays and Vice versa

- To convert List to Array
`Object[] toArray(<Array>)` // <Array> for specifying the return type array data type

Example: MyListArray.java

// This method comes from `ArrayList`

- To Convert Array to List
`static List Arrays.asList(<Array>)` // static method from `Arrays` class

Example: MyArrList.java

Set Interface

Methods of `NavigableSet`

<code>E ceiling(E element)</code>	// Returns the least element $\geq$ E element, else null
<code>E floor(E element)</code>	// Returns the least element $\leq$ E element, else null
<code>Iterator<E> descendingIterator()</code>	// Returns iterator in descending order
<code>NavigableSet<E> descendingSet()</code>	// Returns <code>NavigableSet</code> in descending order
<code>SortedSet<E> headSet(E thisElement)</code>	// Returns a view of this set elements $<$ thisElement
<code>NavigableSet<E> headSet(E thisElement, boolean incl)</code>	// Returns a view of this set elements $<$ or $\leq$ thisElement
<code>SortedSet<E> tailSet(E thisElement)</code>	// Returns a view of this set elements $\geq$ thisElement
<code>NavigableSet<E> tailSet(E thisElement, boolean incl)</code>	// Returns a view of this set elements $>$ or $\geq$ thisElement
<code>SortedSet<E> subSet(E startElement, E endElement)</code>	// Returns a view of this set elements $>$ and $\leq$ element

`NavigableSet<E> subSet(E startElement, boolean startIncl, E endElement, boolean endIncl)`

// Returns a view of this set elements > and < element

`E higher(E element)`

> or null

`E lower(E element)`

< or null

`Iterator<E> iterator()`

Returns iterator in ascending order

`E pollFirst()`

Retrieves and removes the first(lowest)

`E pollLast()`

Retrieves and removes the last(highest)

Single value

Example: TreeSetRep.java

Example: NavigableSetRep.java

Methods of NavigableMap

`Map.Entry<K,V> ceilingEntry(K key)`

highest
>=, null

`Map.Entry<K,V> higherEntry(K key)`

lower
>, null

`K floorKey(K key)`

ceiling
>=, null

`K lowerKey(K key)`

floor
<, null

`Map.Entry<K,V> pollFirstEntry()`

// Removes and Returns First

`Map.Entry<K,V> pollLastEntry()`

// Removes and Returns Last

`SortedMap<K,V> subMap(K startKey, K endKey)`

// startKey >= and < endKey

`NavigableMap<K,V> descendingMap()`

Methods of TreeMap

`SortedMap<K,V> headMap(K endKey)`

// less than the key

`SortedMap<K,V> tailMap(K endKey, boolean include)`

// > the key

`SortedMap<K,V> subMap(K startKey, boolean include, K endKey, boolean include)`

// > the key and < the key

Example: NavigableMapRep.java

Example: SubMapRep.java

Methods of PriorityQueue

`E peek()`

// Retrieves head or null if queue is empty

`E element()`

// Retrieves head or null if queue is empty

`E poll()`

// Retrieves and removes head or null if queue is empty

`E remove()`

// Retrieves and removes head or null if queue is empty

`boolean offer(E element)`

// Inserts the specified element into PriorityQueue

`boolean add(E element)`

// Inserts the specified element into PriorityQueue

`int size()`

// Number of elements

Example: PriorityQueueRep.java

constructor

`PriorityQueue()`

// Ascending

`PriorityQueue(Comparator)`

// custom priority

`PriorityQueue(int initialCapacity, Comparator)`

// Custom priority.

Streams

In Java Introduced in Java 8, the Stream API is used to process collections of objects. A stream is a sequence of objects that supports various methods which can be pipelined to produce the desired result. The features of Java stream are –

A stream is not a data structure instead it takes input from the Collections, Arrays or I/O channels. Streams don't change the original data structure; they only provide the result as per the pipelined methods. Each intermediate operation is lazily executed and returns a stream as a result; hence various intermediate operations can be pipelined. Terminal operations mark the end of the stream and return the result.

Different Operations on Streams

Intermediate Operations

map: The map method is used to map the items in the collection to other objects according to the Predicate passed as argument.

List number = Arrays.asList(2,3,4,5);
List square = number.stream().map(x->xx).collect(Collectors.toList());

filter: The filter method is used to select elements as per the Predicate passed as argument.

List names = Arrays.asList("Reflection", "Collection", "Stream");
List result = names.stream().filter(s->s.startsWith("S")).collect(Collectors.toList());

sorted: The sorted method is used to sort the stream.

List names = Arrays.asList("Reflection", "Collection", "Stream");
List result = names.stream().sorted().collect(Collectors.toList());

Terminal Operations

collect: The collect method is used to return the result of the intermediate operations performed on the stream.

List number = Arrays.asList(2,3,4,5,3);
Set square = number.stream().map(x->xx).collect(Collectors.toSet());

forEach: The forEach method is used to iterate through every element of the stream.

List number = Arrays.asList(2,3,4,5);
number.stream().map(x->xx).forEach(y->System.out.println(y));

reduce: The reduce method is used to reduce the elements of a stream to a single value. The reduce method takes a BinaryOperator as a parameter.

List number = Arrays.asList(2,3,4,5);
int even = number.stream().filter(x->x%2==0).reduce(0,(ans,i)-> ans+i);

Here ans variable is assigned 0 as the initial value and i is added to it.

Assignment

Theory Questions

1. What is the difference between Collection and Map?
2. What is the difference between Collection and Collections?
3. What is the difference between ArrayList and Vector?
4. What is the difference between HashSet, LinkedHashSet and TreeSet? In what situation should we choose HashSet, LinkedHashSet and TreeSet?
5. What is the difference between HashMap, LinkedHashMap and TreeMap? In what situation should we choose HashMap, LinkedHashMap and TreeMap?
6. What is the difference between Comparable and Comparator?

Practical Questions

1. Create a class called Employee that stores his name, department, designation, date of joining and basic salary. Create a menu driven application that performs the following operations :
 - a. Add Employee to List.
 - b. Display Employees in the order of Name.
 - c. Display Employees in the order of basic salary.
 - d. Display Employees in the order of department and designation.
 - e. Exit
2. Create a program that reads contents of a text file word by word and adds it to a TreeMap. The TreeMap should store word as a key and its count as value. At the end it displays number of occurrences of each word in a file.
3. Create a Thread that adds a random number between 100 and 200 to a Queue. Create another thread that executes at random intervals and fetches the number from the queue and display its table. In case the queue is empty the second thread should wait, until it receives a notification from the first thread. Test the output by using PriorityQueue instead of normal queue (LinkedList).
4. Create a class called Vehicles that stores its registration number, chassis number, company, model, owner name, mobiles and his address. Create a menu driven application that performs the following operations :
 - a. Add New Vehicle in Map
 - b. Search Vehicle on the basis of registration number
 - c. Remove Vehicle on the basis of registration number
 - d. Exit
5. Create a class called Actor that stores his name, and date of birth. Create another class called Movies that stores movie name, release date, director and actors acted in that movie. Create a menu driven application with the following options.
 - a. Add New Actor
 - b. Add New Movie
 - c. Add Actors to Movie
 - d. Search Movies
 - e. Search Actors
 - f. Search Movies done by an actor